Design Analysis Report

Smart Hospital Management System

Software Engineering

Prepared by: Abhay Sharma
Roll Number: 202401100200009

Purpose

This report explains the proposed design of the Smart Hospital Management System. It covers the main modules, how information moves through the system, important design choices, possible risks, and recommendations for development. The report has been prepared for an academic Software Engineering project. It describes a proposed design and does not claim that the system has been built, clinically validated, or deployed.

Contents

- Executive summary
- At a glance
- 1. Introduction
- 2. System context and design objectives
- 3. Proposed architecture
- 4. Component and data design
- 5. Key findings
- 6. Design implications and trade-offs
- 7. Risks and mitigations
- 8. Recommendations
- 9. Conclusion
- Appendix A: Assumptions and scope
- Appendix B: Source placeholders

Executive summary

The Smart Hospital Management System (SHMS) is planned as one platform for handling patient registration, appointments, consultation records, prescriptions, vital signs, reminders, and billing. It may also include features such as AI-assisted symptom triage, appointment-slot checking, and alerts. The main aim is to make hospital tasks easier to manage and reduce repeated manual work. Any medical decision, however, must remain with a qualified healthcare professional.

A layered design is suitable for this project because it separates the screens users work with from the rules that run the system and the place where information is stored. The design must protect patient information, give each user only the access they need, prevent two people from booking the same slot, and keep records accurate. It should also handle situations where an external service is unavailable. The safest approach is to build and test the essential features first, then add optional features such as AI, online payments, or teleconsultation.

At a glance

- Main users: patients, doctors, nurses, receptionists/front-desk staff, and hospital administrators.
- Typical process: register the patient → book an appointment → meet the clinician → save notes and prescriptions → send updates → prepare a bill if required.
- Possible smart features: AI-assisted specialty suggestions, appointment-slot checks, alerts for selected vital-sign readings, and automatic reminders.
- Main priorities: patient privacy, appropriate access, clinical safety, reliable appointments, traceable changes, and ease of use.
- Suggested approach: build a secure basic version first and add optional integrations only after testing and approval.

1. Introduction

1.1 Purpose of the report

This report reviews the proposed SHMS design for the Software Engineering subject. It describes what the system will cover, its main modules, how those modules work together, the information they store, and the issues that need attention during development. It is intended to be read with the project's SRS and UML diagrams.

1.2 Problem statement

Hospitals often have to manage appointments, patient details, and clinical notes across different records or manual processes. This can lead to repeated data entry, delayed reminders, and confusion about appointment status. A shared system could make these tasks easier to coordinate. At the same time, it must prevent problems such as unauthorized access, duplicate bookings, incorrect records, and too much reliance on automated suggestions.

1.3 Objectives

- Centralize patient and appointment information for authorized users.
- Support reliable booking, rescheduling, cancellation, and schedule visibility.
- Provide controlled access to consultation notes, prescriptions, and vital-sign history.
- Automate routine notifications and provide clear booking status.
- Offer optional AI-assisted triage as decision support with clear limitations and escalation.
- Provide a maintainable, testable, and secure foundation that can be expanded over time.

2. System context and design objectives

2.1 Stakeholders and responsibilities

Stakeholder	Main needs	Design consideration
Patient	Simple registration, appointment booking, reminders, permitted record access	Accessible UI, privacy, clear status and error messages
Doctor	Schedule visibility, relevant history, consultation notes, prescriptions	Fast workflow, accurate patient matching, clinical audit trail
Nurse	Record vital signs and nursing observations	Measurement units, timestamps, patient identification
Receptionist	Register patients and manage appointments	Limited administrative access without unrestricted clinical permissions
Hospital administrator	Manage users, departments, schedules, and reports	RBAC, audit logs, configurable policies
System operator	Deploy, monitor, back up, and restore the service	Secrets management, monitoring, documented recovery

2.2 Scope boundary

The first version is expected to include:

- Authentication, user roles, and patient profiles.
- Doctor/department directory and appointment lifecycle.
- Medical records, consultation notes, prescriptions, and vital-sign recording.
- Notifications, billing status, and basic administrative reports.
- Audit events and basic operational monitoring.

The following features can be added later if they are required:

- AI-assisted symptom triage and specialty recommendation.
- Online payment gateway and electronic receipts.
- Teleconsultation integration.
- Connected medical devices or IoT vital-sign collection.
- Advanced analytics, predictive reporting, or interoperability with external hospital systems.

3. Proposed architecture

The system is planned in layers so that the user interface, business rules, and stored data are kept separate. This makes the application easier to test and maintain, and it allows an external service to be changed without redesigning the whole system.

Layer / component	Responsibilities	Design notes
Presentation layer	Patient and staff dashboards, forms, appointment calendar, alerts	Responsive interface; display content based on role; validate inputs without relying on client validation alone
API / application layer	Authentication checks, request validation, workflow orchestration	Enforce authorization server-side; consistent error handling and API contracts
Domain services	Appointment, patient, clinical record, billing, notification, scheduling, triage services	Keep business rules in services rather than scattered through UI code
Persistence layer	Store users, appointments, records, prescriptions, vital signs, bills, audit events	Use constraints and transactions for data integrity; protect backups
Integration layer	Email/SMS, payment, video, AI triage, optional device gateways	Use timeouts, retries where safe, status tracking, and fallbacks
Operations and security	Logs, metrics, backups, secrets, deployment configuration	Avoid sensitive health data in routine logs; test restoration and access controls

3.1 High-level request flow

1. The user signs in, and the server checks the account and its permissions.
2. The user carries out an action, such as booking an appointment or adding a consultation note.
3. The API checks the submitted information and confirms that the user is allowed to perform the action.
4. The relevant service applies the system rules and saves the changes safely, using a transaction when needed.
5. For sensitive actions, the system records an audit event. It also arranges any required notifications.
6. The user receives a clear message showing whether the action succeeded or failed, along with the latest status.

This structure keeps each part focused on a clear responsibility. For example, appointment rules should live in the scheduling service rather than being repeated across screens. Sensitive operations should be checked by the server, even if the interface already hides options that a user cannot access. External services should be connected through defined interfaces so that failures in one service do not bring down the main booking and record-keeping functions.

Decision	Rationale	Trade-off
Layered modular architecture	Separates UI, rules, data, and integrations	Requires disciplined interfaces and initial setup
Role-based access control	Limits access according to job responsibility	Permissions must be reviewed and maintained
Transactional appointment booking	Prevents duplicate allocation of a single doctor slot	Concurrency handling adds implementation complexity
Optional AI service boundary	Allows AI to be disabled or replaced without breaking core booking	Requires fallback design and separate validation
Audit trail for sensitive actions	Supports accountability and incident investigation	Adds storage and requires careful access/retention policies
Asynchronous notifications	Avoids delaying booking completion when a message provider is slow	Delivery status may be delayed or unavailable

4. Component and data design

4.1 Main system components

Component	Primary responsibilities	Key interactions
User and Access Management	Login, profile, roles, session handling	All protected modules
Patient Management	Patient demographics and identity matching	Appointments, medical records, billing
Appointment and Scheduling	Availability search, slot reservation, rescheduling, cancellation	Patient, doctor, database, notification service
Clinical Records	Consultation notes, diagnoses, history, prescriptions	Doctor, patient, audit service
Vital-Sign Management	Record and display observations; flag configured thresholds	Nurse/doctor, patient record, notification where configured
AI Triage	Accept symptoms and return advisory guidance	Patient UI, scheduling, optional escalation path
Notification Service	Appointment confirmations, reminders, cancellations	Scheduling service and messaging provider
Billing	Generate bill, record status, issue receipt	Patient, appointment/service, optional payment gateway

Administration and Reporting	Manage departments, staff, schedules, operational summaries	User management, scheduling, reporting data

4.2 Main data entities

Entity	Purpose	Important relationships
User	Shared account identity and role	Parent concept for Patient, Doctor, Nurse, and Administrator profiles
Patient	Patient profile and contact/clinical context	Has appointments, medical records, vital signs, bills, and triage results
Doctor	Clinician profile, specialization, availability	Belongs to a department; attends appointments; creates clinical entries and prescriptions
Department	Hospital unit or specialty group	Contains or is associated with multiple doctors
Appointment	Scheduled patient visit	References one patient and one doctor; may link to a bill and clinical record
MedicalRecord	Consultation notes and history	Belongs to a patient and records the author; may link to an appointment
Prescription	Medication instructions authored by a clinician	References patient, doctor, and relevant clinical record
VitalSigns	Time-stamped measurements	Belongs to a patient and identifies the recorder or device
TriageResult	Advisory specialty/urgency suggestion	Associated with a patient/session subject to data-minimization policy
Bill / Payment	Charge and payment tracking	Belongs to a patient and may reference an appointment or service
Notification	Delivery state for alerts/reminders	References recipient and triggering event
AuditEvent	Record of sensitive access or changes	References actor, action, resource, timestamp, and outcome

4.3 Keeping data accurate and consistent

- Give each record a unique identifier and check that linked records exist.
- Reserve appointment slots in a way that prevents two users from booking the same slot at the same time.
- Use consistent units and timestamps, and decide which time zone the hospital will use for display.
- Keep a record of who added or changed clinical information and when the change was made.
- Keep operational logs separate from clinical records. Avoid putting unnecessary symptom or health details in logs.
- Decide how long records will be kept and how corrections, exports, and deletion requests will be handled before using real patient data.

5. Key findings

The review highlights a few design choices that will have the biggest effect on whether the system is safe and dependable. These points come from analysing the proposed design; they are not results from live-system testing.

Finding	Observation	Priority
Security is foundational	The platform handles identity and health-related information, so authorization and data minimization must be built into the core architecture.	Critical
Appointment consistency is a core correctness issue	Concurrent booking requests can otherwise allocate the same slot more than once.	High
AI must be bounded	Symptom analysis can be misleading; the interface must communicate uncertainty and preserve a route to professional care.	Critical
External integrations are failure points	SMS, payment, video, and AI providers can time out or become unavailable.	High
Clinical data needs	Author, timestamp, and change history support accountability and help	High

provenance	users interpret records.	
Usability affects adoption	Patients and staff need concise workflows, understandable status messages, and accessible interfaces.	Medium
Scope needs phased delivery	Implementing every smart feature at once increases complexity and makes verification harder.	High

6. Design implications and trade-offs

Theme	Implication	Design response
Privacy vs. convenience	A single shared record is useful but increases exposure risk.	Apply least privilege, patient-specific authorization, session controls, audit events, and data minimization.
Automation vs. clinical responsibility	Automated triage can help route patients but may be wrong or incomplete.	Use advisory language, emergency escalation guidance, documented limitations, and human clinical decision-making.
Immediate vs. reliable notification	Waiting for an SMS/email response can slow booking.	Persist booking first, then send notification asynchronously and track delivery when possible.
Rich features vs. delivery risk	AI, teleconsultation, devices, and payments add external dependencies.	Use a phased roadmap and keep optional services behind stable interfaces.
Analytics vs. data minimization	Reports can improve operations but may expose sensitive details.	Prefer aggregated reports and restrict row-level exports to authorized roles.
Availability vs. complexity	High availability and disaster recovery add cost and operational work.	Set realistic service targets and test backups and restoration early.

7. Risks and mitigations

Risk	Potential impact	Mitigation / control	Priority
Unauthorized record access	Privacy breach and loss of trust	Server-side RBAC, patient ownership checks, audit logs, security tests	Critical
Duplicate or conflicting appointments	Patient inconvenience and schedule disruption	Atomic slot reservation, unique constraints, concurrency tests	High
Incorrect AI triage output	Inappropriate reassurance or delay in care	Non-diagnostic framing, safe escalation, clinician oversight, testing and monitoring	Critical
Third-party service outage	Missing reminders or unavailable optional feature	Timeouts, retries where safe, status tracking, graceful fallback	High
Data loss or corruption	Loss of clinical and operational history	Backups, access controls, integrity checks, restore drills	Critical
Overly broad administrator privileges	Excessive access to clinical data	Separate technical administration from clinical authorization; review permissions	High
Poor usability or accessibility	Errors, abandonment, increased staff workload	Task-based usability testing, accessible labels, clear feedback	Medium
Unclear retention/consent rules	Compliance and operational risk	Define policy with institution and applicable legal guidance before real-data use	High

8. Recommendations

7. Confirm scope with stakeholders. Decide which modules are mandatory for the first release and label AI triage, payment, teleconsultation, and device integration as optional unless specifically required.
8. Implement security before clinical features. Establish role permissions, patient ownership checks, secure credential handling, session management, audit logging, and safe error handling early.
9. Prioritize core workflows. Build and test registration, appointment booking, cancellation/rescheduling, doctor consultation, clinical notes, and notifications before advanced analytics.

10. Protect appointment consistency. Use database transactions or equivalent concurrency controls, then test simultaneous booking attempts against the same slot.
11. Treat AI triage as a separate, optional decision-support module. Document intended use and limitations, test representative scenarios, show clear disclaimers, and provide a fallback and escalation route.
12. Design data governance with the hospital. Define who may view or change each record type, how consent is captured, how long data is retained, and how exports and corrections are handled.
13. Test failure paths, not only happy paths. Simulate notification-provider failures, AI timeouts, invalid input, expired sessions, database errors, and interrupted booking requests.
14. Use measurable acceptance criteria. Define performance targets, supported concurrent users, recovery targets, and usability checks before the final demonstration.
15. Keep documentation synchronized. Update the SRS, class diagram, sequence/activity diagrams, API contracts, test cases, and this report when requirements change.
16. Validate before real-world use. The academic prototype should not be used for actual clinical decision-making or real patient data until appropriate clinical, security, privacy, and operational reviews are completed.

9. Conclusion

The proposed Smart Hospital Management System could bring patient administration and selected clinical tasks together in one place. To make it dependable, the design needs clear user permissions, reliable appointment booking, a record of important changes, and a plan for handling failures in connected services. Smart features may be useful, but they should be introduced carefully and should not replace professional medical judgement. The next step is to confirm the requirements with the intended users, build the core features, and test them against clear acceptance criteria before adding more functionality.

Appendix A: Assumptions and scope notes

- This report describes a proposed academic design; it does not assert that the system has been implemented or tested.
- The deployment jurisdiction and applicable healthcare/privacy requirements must be confirmed by the institution.
- Specific technology choices are intentionally not fixed; the architecture can be implemented using an appropriate web/mobile stack and database.
- AI triage, payments, teleconsultation, and connected medical devices are optional until confirmed in the project scope.
- Any numerical service-level targets should be finalized based on the actual hosting environment and expected workload.

Appendix B: Source placeholders

Add the actual sources used for the project before submitting the report. The items below are placeholders and should be checked or replaced as needed.

- Project team. Smart Hospital Management System — Software Requirements Specification (SRS). Version 1.0, 2026.
- Project team. Smart Hospital Management System — UML Class Diagram. Version 1.0, 2026.
- Project team. Smart Hospital Management System — Sequence and Activity Diagrams. Version 1.0, 2026.
- Relevant software engineering textbook or course notes prescribed by the institution.
- Applicable healthcare privacy, security, and accessibility guidance for the deployment jurisdiction.